

Exercise 2: Implementing the Factory Method Pattern

Scenario:

You are developing a document management system that needs to create different types of documents (e.g., Word, PDF, Excel). Use the Factory Method Pattern to achieve this.

Steps:

1. Create a New Java Project:

- o Create a new Java project named `FactoryMethodPatternExample`.

2. Define Document Classes:

- o Create interfaces or abstract classes for different document types such as `WordDocument`, `PdfDocument`, and `ExcelDocument`.

3. Create Concrete Document Classes:

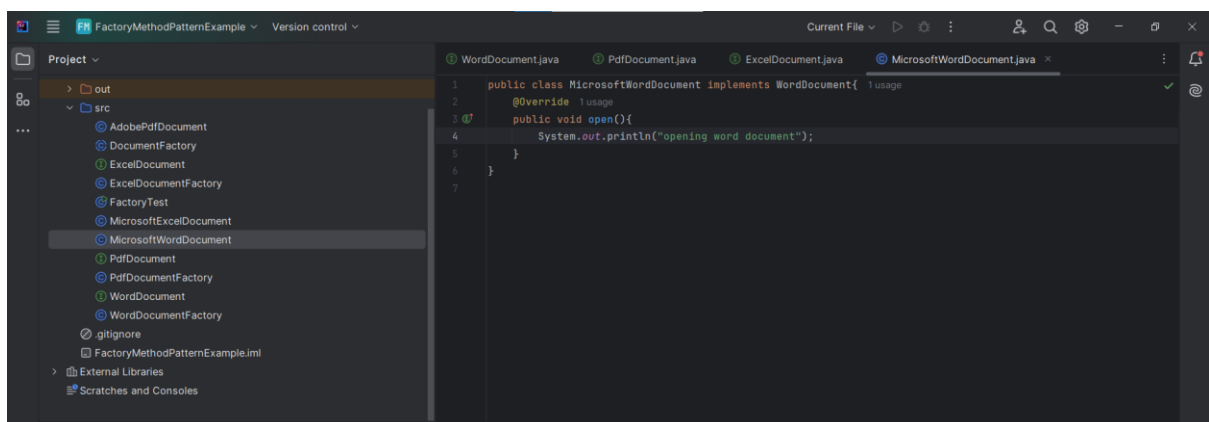
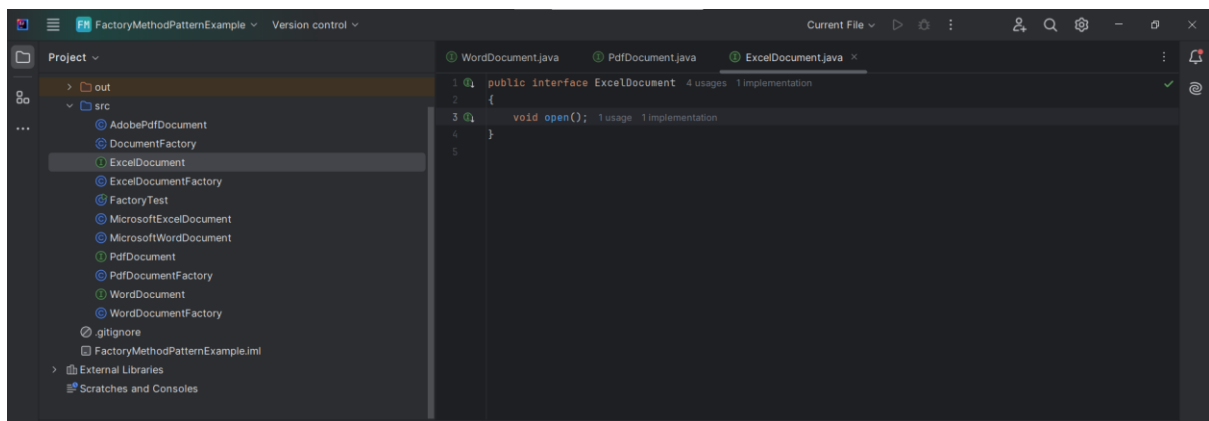
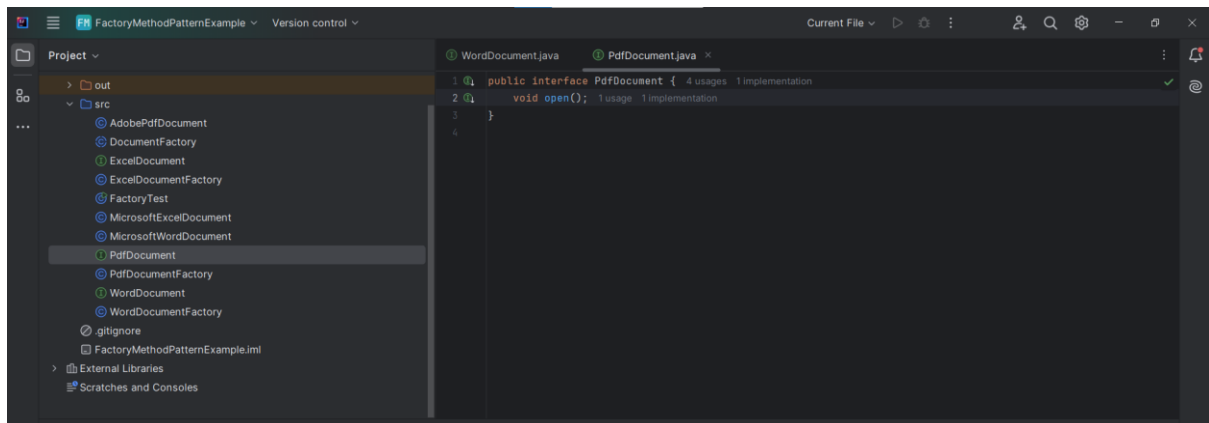
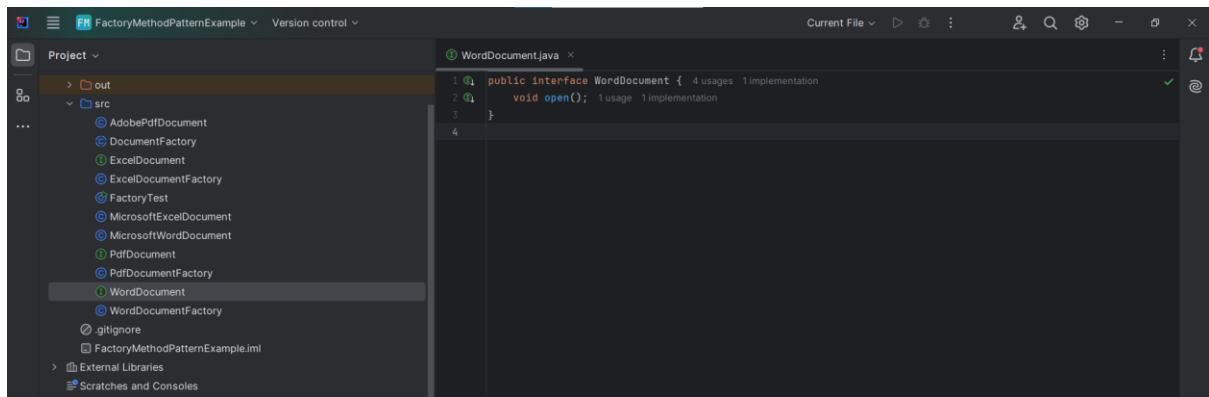
- o Implement concrete classes for each document type that implements or extends the above interfaces or abstract classes.

4. Implement the Factory Method:

- o Create an abstract class `DocumentFactory` with a method `createDocument()`.
- o Create concrete factory classes for each document type that extends `DocumentFactory` and implements the `createDocument()` method.

5. Test the Factory Method Implementation:

- o Create a test class to demonstrate the creation of different document types using the factory method.



This screenshot shows the IDE with the `AdobePdfDocument.java` file open. The file implements the `PdfDocument` interface. The code is as follows:

```
1 public class AdobePdfDocument implements PdfDocument { 1 usage
2     @Override 1 usage
3     public void open() {
4         System.out.println("opening pdf document");
5     }
6 }
7
```

This screenshot shows the IDE with the `MicrosoftExcelDocument.java` file open. The file implements the `ExcelDocument` interface. The code is as follows:

```
1 public class MicrosoftExcelDocument implements ExcelDocument { 1 usage
2     @Override 1 usage
3     public void open() {
4         System.out.println("opening Excel document");
5     }
6 }
7
```

This screenshot shows the IDE with the `DocumentFactory.java` file open. The file is an abstract class that defines the `createDocument()` method. The code is as follows:

```
1 public abstract class DocumentFactory { 6 usages 3 inheritors
2     public abstract Object createDocument(); 3 usages 3 implementations
3 }
4
```

This screenshot shows the IDE with the `WordDocumentFactory.java` file open. The file extends the `DocumentFactory` class and implements the `createDocument()` method. The code is as follows:

```
1 public class WordDocumentFactory extends DocumentFactory { 1 usage
2     @Override 3 usages
3     public WordDocument createDocument() {
4         return new MicrosoftWordDocument();
5     }
6 }
7
```

```
1 public class PdfDocumentFactory extends DocumentFactory { 1 usage
2
3     @Override 3 usages
4     public PdfDocument createDocument() {
5         return new AdobePdfDocument();
6     }
7
8 }
```

```
1 public class ExcelDocumentFactory extends DocumentFactory { 1 usage
2
3     @Override 3 usages
4     public ExcelDocument createDocument() {
5         return new MicrosoftExcelDocument();
6     }
7
8 }
```

```
1 public class FactoryTest {
2     public static void main(String[] args) {
3
4         // Word Document
5         DocumentFactory wordFactory = new WordDocumentFactory();
6         WordDocument word = (WordDocument) wordFactory.createDocument();
7         word.open();
8
9         // PDF Document
10        DocumentFactory pdfFactory = new PdfDocumentFactory();
11        PdfDocument pdf = (PdfDocument) pdfFactory.createDocument();
12        pdf.open();
13
14        // Excel Document
15        DocumentFactory excelFactory = new ExcelDocumentFactory();
16        ExcelDocument excel = (ExcelDocument) excelFactory.createDocument();
17        excel.open();
18    }
19
20 }
```

OUTPUT

```
opening word document
opening pdf document
opening Excel document

Process finished with exit code 0
```